PyTorch Conference EU 2026

# Recursive Language Models (RLMs): Scaling to Infinite Context via Programmatic Decomposition

Rudraksh Karpe[1] Shivay Lamba[2]

[1]Simplismart [2]Qualcomm

## OVERVIEW

Recursive Language Models (RLMs), developed at MIT CSAIL, enable any LLM to process **10M+ tokens** without retraining. An RLM is a thin wrapper around a language model: from the user's perspective, `rlm.completion(messages)` is a drop-in replacement for `gpt5.completion(messages)`. Under the hood, the prompt lives as data in an external **Python REPL** and the model writes code to peek, chunk, grep, and recursively call itself on smaller slices.

Unlike agents that decompose by *task*, RLMs decompose by *context* - the LM decides how to break down its input. **RLM(GPT-5-mini)** outperforms GPT-5 by **+114%** on OOLONG at comparable cost. **RLM-Qwen3-8B**, a post-trained open model, achieves **+28.3%** median gain over its base.

## MOTIVATION

- **Context rot is real.** As conversation length grows, models get measurably worse - even within their context window. On OOLONG, GPT-5 scores drop as context doubles from 132K to 263K tokens.
- Summarization heuristics assume early context is forgettable - wrong for tasks needing $\Omega(|P|)$ work (e.g. counting, labeling thousands of entries).
- **No symbolic handle:** the prompt exists only as tokens inside the model - no way to index, slice, grep, or loop over it programmatically.
- **Verbal sub-delegation:** prior agent scaffolds verbalize a few sub-tasks; they cannot launch $\Omega(|P|)$ programmatic sub-calls in a tight loop.
- **Output bounded by K:** direct generation cannot produce responses longer than the context window.

## CORE IDEA

An RLM makes three design choices missing from existing scaffolds:

1. **Prompt as environment data.** The full prompt P lives *outside* the transformer as a variable in a persistent Python REPL.
2. **Constant-size root context.** The root language model sees only short metadata (length, prefix, stdout summaries) - never the raw prompt tokens.
3. **Symbolic recursion.** The model writes Python that peeks, chunks, searches, and calls `sub_RLM(…)` inside loops. When `Final` is set in the REPL, iteration stops and its value is returned.

## RLM VS. EXISTING METHODS

| Dimension | Base LLM | RAG (BM25) | Summary Agent | Sub-Agent | RLM |
|---|---|---|---|---|---|
| Prompt handling | In-context | Retrieved chunks | Iterative compress | Verbal split | External REPL |
| Max input | ~128K | Index any | ~128K | ~128K | 10M+ |
| Dense access | No | Sparse | Lossy | Partial | Yes |
| Sub-tasks | None | None | None | Verbal | Code loops |
| $\Omega(\|P\|)$ calls | No | No | No | No | Yes |
| Output limit | K tokens | K tokens | K tokens | K tokens | Unbounded |
| Retraining | No | No | No | No | No (opt. SFT) |
| Cost profile | Fixed | Fixed + idx | Linear | Linear | Variable |

K = model context window. RAG retrieves relevant chunks but cannot densely process every token. Summary agents lose detail on compaction. Only RLMs support true programmatic recursion over the full input.

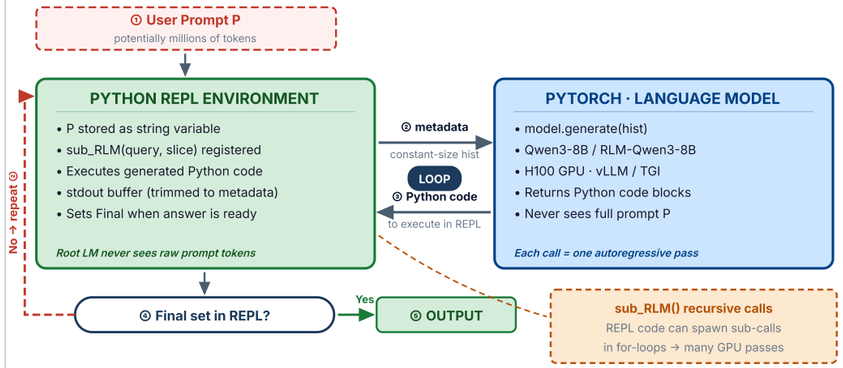
## RELATED WORK

- **Context compaction** (Codex CLI, summary agents): iteratively summarizes context when full. Loses dense detail; presumes forgettable content.
- **Retrieval agents** (CodeAct + BM25): index input with a retriever. Cannot densely process every line - limited by underlying model window.
- **Sub-agent delegation** (Claude sub-agents, Context Folding): verbal delegation, not programmatic. Cannot loop $\Omega(|P|)$ times.
- **RLMs** combine the REPL approach with symbolic recursion, enabling dense access to arbitrarily long inputs via code.

## ARCHITECTURE & METHODS

Given prompt P, an RLM initialises a Python REPL with P as a variable and a `sub_RLM()` hook. The loop: model generates code → REPL executes → constant-size metadata returns → repeat until `Final` is set.

RLM inference loop - architecture view




### Where PyTorch Fits in the Stack

The recursive control flow is *programmatic* (Python), not neural. **PyTorch powers the language model** - every root step and every sub-call is one autoregressive `generate()` pass on GPU.

System stack - where PyTorch sits

| | |
|---|---|
| REPL | Persistent Python env. Stores P as a variable, intermediate chunks, `sub_RLM()` hook, stdout buffer. |
| ORCHESTRATOR | RLM loop: assemble `hist` (constant-size metadata only) → call the language model → execute returned code → trim stdout → repeat. |
| PYTORCH | `transformers` / vLLM / TGI load **Qwen3-8B** or **RLM-Qwen3-8B** weights. Each root step & each sub-call = one `model.generate()` on H100 GPU. |
| TRAINING | SFT via **prime-rl** (PyTorch): distill 1K filtered trajectories from Qwen3-Coder-480B RLM → **RLM-Qwen3-8B**. ~48 H100-hours. |

**Key distinction:** recursion = program control flow in the REPL, not extra transformer layers. PyTorch handles the language model's inference and the SFT training loop.

## EMERGENT RLM STRATEGIES

Without explicit training, RLMs discover reusable patterns for handling long context:

| Strategy | What the Root LM Does | When Used |
|---|---|---|
| Peeking | Reads first N chars to discover structure | Start of every RLM call |
| Grepping | Regex / keyword search to narrow lines | Needle-in-haystack, filtering |
| Partition + Map | Chunks context, sub_RLM() per chunk | Dense labeling, counting |
| Summarize | Condenses sub-call outputs to aggregate | Multi-hop reasoning |
| Programmatic | Writes code to process diffs / compute | Long-output tasks |

These strategies compose: peek first, grep to filter, then partition + map sub-calls over filtered slices.

## KEY FINDINGS

- **RLMs ≠ agents:** agents decompose by *task*; RLMs decompose by *context*.
- **Prefix caching:** KV-cache reuse cuts latency **25-30%** with no accuracy loss.
- **SFT 3× more efficient: 3.2× fewer tokens**, **3.9× fewer iterations**, +4 pts accuracy.
- **No context rot:** stable at 10M+ tokens; root LM never sees full context.

## OPTIMIZING RLM INFERENCE WITH vLLM

**The problem:** RLM's recursive loop generates **7-8 iterations** per query on base Qwen3-8B, each requiring a full autoregressive pass. Sub-calls within iterations run sequentially. On a vanilla vLLM setup, this means **~60s per query** - most of it redundant KV recomputation over shared prompt prefixes.

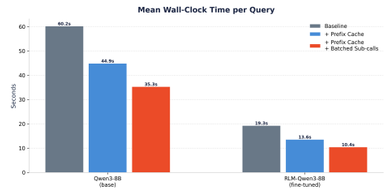
- **Prefix caching (-25% latency):** RLM iterations share a long, identical system prompt + REPL context prefix. Enabling vLLM's `--enable-prefix-caching` lets the KV-cache persist across calls, eliminating redundant prefill for every iteration and sub-call. Baseline 60.2s → **44.9s**.
- **Batched sub-calls (-41% total):** Within each iteration, independent `sub_RLM()` invocations (e.g. partition-map over chunks) can run concurrently. Setting concurrency to 4 overlaps GPU inference for parallel sub-calls. Baseline 60.2s → **35.3s**.
- **SFT amplifies gains:** RLM-Qwen3-8B (distilled) needs only **1.9 iterations** vs. 7.5, so even the baseline is 3× faster. With both optimizations: 19.3s → **10.4s** (-46%).

Setup: 1× H100 80GB · vLLM · `bfloat16` · 32K max len · 0.90 GPU mem · RLM: 30 max iters, depth 1 · OOLONG-synth (20 samples)

## BENCHMARK RESULTS

Results from the three vLLM configs above, comparing base Qwen3-8B vs. RLM-Qwen3-8B (SFT) on OOLONG-synth (20 samples). Best config achieves **up to 46% latency reduction** on RLM-Qwen3-8B.

| Model | Config | Score | Latency | VRAM (GiB) | Tokens | Iters | Sub-calls |
|---|---|---|---|---|---|---|---|
| **Qwen3-8B (base)** | Baseline | 36.8% | 60.2s | 71.9 | 47,332 | 7.5 | 1.5 |
| **Qwen3-8B (base)** | + Prefix Cache | 36.8% | 44.9s | 71.9 | 47,332 | 7.5 | 1.5 |
| **Qwen3-8B (base)** | + Cache + Batch | 36.8% | 35.3s | 71.9 | 47,332 | 7.5 | 1.5 |
| **RLM-Qwen3-8B (SFT)** | Baseline | 40.8% | 19.3s | 71.9 | 14,659 | 1.9 | 2.5 |
| **RLM-Qwen3-8B (SFT)** | + Prefix Cache | 40.8% | 13.6s | 71.9 | 14,659 | 1.9 | 2.5 |
| **RLM-Qwen3-8B (SFT)** | + Cache + Batch | 40.8% | 10.4s | 71.9 | 14,659 | 1.9 | 2.5 |

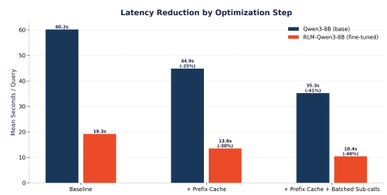


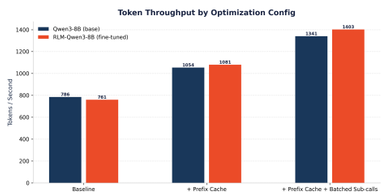

Task Accuracy (OOLONG-synth)

| | Baseline | Prefix Cache | Cache + Batched |
|---|---|---|---|
| Qwen3-8B (base) | 36.8% | 36.8% | 36.8% |
| RLM-Qwen3-8B (fine-tuned) | 40.8% | 40.8% | 40.8% |

### vLLM Optimization Impact

**Prefix caching** exploits the fact that every RLM iteration shares a long, identical system-prompt + REPL-context prefix. Enabling `--enable-prefix-caching` lets vLLM persist the KV-cache across calls, eliminating redundant prefill and cutting wall-clock time by **25%**. **Batched sub-calls** go further: independent `sub_RLM()` invocations within an iteration run concurrently (concurrency = 4), overlapping GPU inference to reach a combined **-41%** latency reduction. The SFT model amplifies both gains—requiring only **1.9** iterations vs. 7.5—so the full optimization stack delivers **10.4s/query** (down from 60.2s baseline).




**References** [1] A. L. Zhang, T. Kraska, O. Khattab. "Recursive Language Models." arXiv 2512.24601v2, 2025. arxiv.org/html/2512.24601v2 [2] Reference Inference Implementation: github.com/alexzhang13/rlm [3] RLM-Qwen3-8B: huggingface.co/mit-oasys/rlm-qwen3-8b-v0.1 | Qwen3-8B: huggingface.co/Qwen/Qwen3-8B

BENCHMARKING POC GITHUB REPO